MindLink

Backend Completion Guide

A step-by-step guide to completing your Node.js + MongoDB backend

Current State

File	Status	Notes
models/user.ts	■ Done	nome, genero, data_nascimento, email, password, tipo
models/psicologo.ts	■ Done	user (ref), especialidade
models/paciente.ts	■ Done	user (ref), psicologo (ref), doenca, formulario
routes/UserRoutes.ts	■ Done	register, login, get all, get by id
models/consulta.ts	■ Missing	Needs to be created
models/questionario.ts	■ Missing	Needs to be created
models/desafio.ts	■ Missing	Needs to be created
models/mensagem.ts	■ Missing	Needs to be created
routes/PsicologoRoutes.ts	■ Missing	Needs to be created
routes/PacienteRoutes.ts	■ Missing	Needs to be created
auth/VerifyToken.ts	■■ Incomplete	Exists but not wired up
Password hashing	■■ Missing	Passwords stored in plain text — needs bcrypt

STEP 1 Hash Passwords with bcrypt

Right now passwords are stored in plain text in MongoDB — this is a security risk. Before going further, install bcrypt and update UserRoutes.ts to hash on register and compare on login.

Install:

```
npm install bcryptjs
```

```
npm install --save-dev @types/bcryptjs
```

Update UserRoutes.ts — Register route:

```
const bcrypt = require('bcryptjs'); // Inside POST /register: const hashedPassword = await bcrypt.hash(password, 10); const user = new User({ nome, genero, data_nascimento, email, password: hashedPassword, tipo });
```

Update UserRoutes.ts — Login route:

```
// Replace User.findOne({ email, password }) with: const user = await User.findOne({ email }); if (!user) return res.status(401).json({ error: 'Email ou password incorretos' }); const passwordMatch = await bcrypt.compare(password, user.password); if (!passwordMatch) return res.status(401).json({ error: 'Email ou password incorretos' });
```

STEP 2 Wire Up JWT Authentication

You already have auth/VerifyToken.ts — now use it. JWT tokens protect routes so only logged-in users can access them.

Install:

```
npm install jsonwebtoken npm install --save-dev @types/jsonwebtoken
```

Return a token on login (UserRoutes.ts):

```
const jwt = require('jsonwebtoken'); const token = jwt.sign( { id: user._id, tipo: user.tipo }, process.env.JWT_SECRET || 'secret_key', { expiresIn: '7d' } ); res.json({ token, user: safeUser });
```

Protect a route using VerifyToken middleware:

```
const { verifyToken } = require('../auth/VerifyToken'); // Add verifyToken as middleware before your route handler: router.get('/profile', verifyToken, async (req: any, res: any) => { res.json({ user: req.user }); });
```

Note: Store your JWT secret in a .env file and add .env to .gitignore. Install dotenv: npm install dotenv

STEP 3 Create PsicologoRoutes.ts and PacienteRoutes.ts

Create a routes file for each entity, mirroring the structure of UserRoutes.ts. Each file should have at minimum: create, get all, get by ID.

Route	Method	Description
POST /api/psicologos	POST	Create a new psicologo (links to an existing User)
GET /api/psicologos	GET	List all psicologos
GET /api/psicologos/:id	GET	Get a single psicologo by ID
POST /api/pacientes	POST	Create a new paciente (links to User + Psicologo)
GET /api/pacientes	GET	List all pacientes
GET /api/pacientes/:id	GET	Get a single paciente by ID

Register both in server.ts:

```
const { psicologoRoutes } = require('./routes/PsicologoRoutes'); const { pacienteRoutes } = require('./routes/PacienteRoutes'); app.use('/api/psicologos', psicologoRoutes); app.use('/api/pacientes', pacienteRoutes);
```

STEP 4 Create the Remaining Models

Your schema defines four more entities. Create a model file for each one in the models/ folder.

models/consulta.ts

- id_paciente — ObjectId, ref: 'Paciente'
- id_medico — ObjectId, ref: 'Psicologo'
- data — Date
- estado — String, enum: ['agendada', 'realizada']

models/questionario.ts

- id_paciente — ObjectId, ref: 'Paciente'
- data — Date
- humor — String (or Number 1–5)
- sintomas — String
- notas — String

models/desafio.ts

- id_paciente — ObjectId, ref: 'Paciente'
- id_medico — ObjectId, ref: 'Psicologo'
- titulo — String
- descricao — String
- tipo — String, enum: ['diario', 'semanal']

- data_criacao — Date
- data_inicio — Date
- data_fim — Date
- estado — String, enum: ['pendente', 'concluido']

models/mensagem.ts

- id_paciente — ObjectId, ref: 'Paciente'
- id_medico — ObjectId, ref: 'Psicologo'
- mensagem — String
- data — Date
- hora — String

STEP 5 Create Routes for Remaining Entities

Each model needs its own routes file. Create ConsultaRoutes.ts, QuestionarioRoutes.ts, DesafioRoutes.ts, and MensagemRoutes.ts — then register them all in server.ts.

Entity	Base Path	Suggested Routes
Consulta	/api/consultas	POST (create), GET (all), GET /:id, PUT /:id (update estado)
Questionario	/api/questionarios	POST (create), GET (all), GET /:id
Desafio	/api/desafios	POST (create), GET (all), GET /:id, PUT /:id (update estado)
Mensagem	/api/mensagens	POST (send), GET (all for a patient/doctor)

STEP 6 Test the Full Flow in Bruno

Once all routes are in place, test the complete end-to-end flow in Bruno in this order:

#	Action	Endpoint	Notes
1	Register a User (tipo: psicologo)	POST /api/users/register	Save the returned _id
2	Create a Psicologo	POST /api/psicologos	Pass the User _id in the body. Save psicologo _id
3	Register another User (tipo: paciente)	POST /api/users/register	Save the returned _id
4	Create a Paciente	POST /api/pacientes	Pass user _id + psicologo _id in the body
5	Login as the psicologo	POST /api/users/login	Save the returned JWT token
6	Create a Consulta	POST /api/consultas	Pass paciente _id + psicologo _id + data + estado
7	Create a Desafio	POST /api/desafios	Assign a challenge to a patient
8	Create a Questionario	POST /api/questionarios	Patient fills in their daily questionnaire
9	Send a Mensagem	POST /api/mensagens	Patient or doctor sends a message

STEP 7 Final Checklist Before Frontend

- ■ All models created and match the database schema
- ■ All route files created and registered in server.ts
- ■ Passwords hashed with bcrypt (never stored in plain text)
- ■ JWT auth working — login returns a token
- ■ Protected routes require a valid token
- ■ All routes tested in Bruno with valid responses
- ■ .env file used for secrets (JWT_SECRET, MongoDB URI)
- ■ .env added to .gitignore
- ■ Error handling returns clear messages (400, 401, 404, 500)

Once all items above are checked off, your backend is ready to connect to the frontend.